

Tracing Pointer Movements in `segregateCorrectly`

Let's walk through the code step by step on this sample array:

Copy

```
[12, -7, -5, 70, -3, 15]
```

We'll show at each major step:

1. The positions of `left` (L) and `right` (R)
2. How each inner `while` moves pointers
3. The swap (if any)
4. An ASCII "snapshot" of the array with pointers

Initial State

- Array: `[12, -7, -5, 70, -3, 15]`
- `left = 0`
- `right = 5`

ASCII:

Copy

```
indices:  0   1   2   3   4   5
array:    [12, -7, -5, 70, -3, 15]
           L                       R
```

Iteration 1

1. **Move** `left` until we find a non-negative number
 - `arr[0] = 12` is non-negative $\Rightarrow$ stop.
 - `left` remains at 0.
2. **Move** `right` until we find a negative number
 - `arr[5] = 15` $\geq 0 \Rightarrow$ decrement `right` to 4
 - `arr[4] = -3` $< 0 \Rightarrow$ stop.
 - `right` is now 4.
3. **Swap** since `left < right` ($0 < 4$)

- Swap `arr[0]` and `arr[4]`
- New array: `[-3, -7, -5, 70, 12, 15]`
- Increment `left` to 1, decrement `right` to 3

ASCII after swap:

Copy

```
indices:  0    1    2    3    4    5
array:    [-3,  -7,  -5, 70, 12, 15]
           L              R
```

Step	left before	right before	Action	left after	right after
Iteration 1	0	5	move R→4 (15≥0), swap [0↔4]	1	3

Iteration 2

- Move** `left` until non-negative
 - `arr[1] = -7 < 0 ⇒ left →2`
 - `arr[2] = -5 < 0 ⇒ left →3`
 - `arr[3] = 70 ≥ 0 ⇒ stop at 3`
- Move** `right` until negative
 - `arr[3] = 70 ≥ 0 ⇒ right →2`
- Now `left = 3` and `right = 2`. Since `left < right` is **false**, we do **not** swap and exit the loop.

ASCII at loop exit (pointers have crossed):

Copy

```
indices:  0    1    2    3    4    5
array:    [-3,  -7,  -5, 70, 12, 15]
           R    L
```

Step	left before	right before	Action	left after	right after
Iteration 2	1	3	move L→3 (two negatives), R→2	3	2



Where Pointers Cross

- The loop terminates the instant `left` is no longer less than `right`.
- On Iteration 2, after advancing both pointers, we reach `left = 3` and `right = 2`.
- At that moment the condition `while (left < right)` fails and we break out.

Final Result

Copy

```
[-3, -7, -5, 70, 12, 15]
```

All negative numbers have been pushed left of the crossing point, and all non-negative numbers to the right.

Quick Visual Summary

Copy

```
Initial:      [12, -7, -5, 70, -3, 15]
               L               R
```

```
After Swap #1: [-3, -7, -5, 70, 12, 15]
                L               R
```

```
Pointers cross: [-3, -7, -5, 70, 12, 15]
                  R   L   ← loop ends here
```

— By following this trace you can see exactly how the two-pointer scheme partitions negatives and non-negatives in one pass, swapping in place until the pointers cross.